INFRAPULSE

System Architecture, Data Design and UX Specification

SDG 9 — Industry, Innovation and Infrastructure

ARCHITECTURE & DESIGN DOCUMENT

Build-ready specification for an AI-assisted development workflow

Core proposition: Infrastructure availability does not guarantee reliable service. InfraPulse combines availability, service-performance evidence, accessibility, and population impact to help identify reliability gaps and compare interventions.

Version 1.0 | Prepared 9 October 2026 | Hackathon MVP with production-minded extension path

1. Architecture goals

The architecture should be simple enough to finish as a 24-hour MVP and modular enough to add verified feeds later. Keep deterministic scoring and ranking in the backend domain layer. Treat external datasets and AI text generation as optional adapters, not core dependencies.

2. Logical architecture

```
Data sources (CSV/GeoJSON/public data/manual verification) → ingestion and validation → canonical village/indicator model → normalization and scoring service → gap and priority services → REST API → React web app (dashboard, map, village profile, scenario simulator, methodology) → exports and audit logs.
```

Layer	Responsibilities	Suggested implementation
Presentation	Dashboard, map, filters, table, profile, scenario comparison, methodology and source display.	React + Vite + TypeScript; Tailwind or CSS modules; React Router.
Map	Village points/polygons, choropleth categories, selected feature, legend, filters.	Leaflet + GeoJSON for MVP; MapLibre if vector-tile needs arise.
API	Serve village list/detail, map features, rankings, score metadata, scenarios, imports and exports.	FastAPI + Pydantic schemas.
Domain services	Normalization, score aggregation, coverage, gap detection, priority calculation, scenario recalculation.	Pure Python modules with unit tests; avoid embedding formulas in routes.
Persistence	Village identity/geography, indicators, sources, score versions, reports and audit events.	PostgreSQL + PostGIS for multi-user growth; SQLite acceptable for local demo.
Ingestion	CSV/GeoJSON validation, duplicate handling, coordinate validation, provenance capture.	Pandas/geopandas only where needed; small CSV parser is enough for MVP.
Operations	Environment config, logs, health checks, deployment and data backup.	Vercel frontend; Render/Fly.io or equivalent API; managed Postgres if available.

3. Recommended repository structure

```
infrapulse/ frontend/ src/ app/ # routes and application shell components/ # reusable cards, tables, filters, legend features/ dashboard/ map/ village-profile/ rankings/ simulator/ methodology/ lib/ # API client, formatting, validation types/ public/data/ # small demo GeoJSON only if static fallback is needed backend/ app/ main.py api/routes/ core/config.py domain/ normalization.py scoring.py gap_analysis.py prioritization.py simulation.py schemas/ repositories/ services/ tests/ data/ # seed/demo data with README and provenance docs/ PRD.md ARCHITECTURE.md TASKS.md DATA_DICTIONARY.md scripts/ README.md .env.example
```

4. Data model / relational design

Entity	Key fields	Notes
Village	id UUID/serial; external_code; name; state; district; block; population; population_year; geom/lat/lon.	Use stable internal ID; keep external codes in a separate field.
DataSource	id; name; url; publisher; licence; retrieval_date; notes.	One source may support many indicator observations.
IndicatorObservation	id; village_id; dimension; indicator_key; raw_value; unit; observed_at; period_start/end; source_id; evidence_type; confidence; verification_status; spatial_resolution; notes.	Store observations as facts; avoid overwriting history.
ScoringConfig	version; dimension_weights JSON; normalization_rules JSON; category_thresholds JSON; min_coverage; created_at.	Every score must reference a config version.
VillageScore	id; village_id; config_version; dimension_scores JSON; overall_score; coverage_percent; category; calculated_at.	Can be materialized or computed on demand for MVP.
PriorityResult	id; village_id; version; deficit_factor; population_factor; criticality_factor; alternative_factor; confidence_factor; priority_score; reasons JSON.	Reasons must be based on structured factors.
Scenario	id; name; village_id; baseline_score_id; overrides JSON; estimated_cost; assumptions; created_at.	Scenario overrides must not mutate observations or baseline scores.
FieldReport (optional)	id; village_id; dimension; report_text; created_at; evidence_url; verification_status; reporter_private_ref (restricted).	Do not expose reporter identity in public views.

5. API contract

Method / route	Purpose	Response behavior
GET /health	Deployment health check.	200 with service status and version.
GET /api/v1/summary	Dashboard totals and distribution.	Filters accepted consistently with village list.
GET /api/v1/villages	Paginated village list; filters: district, category, dimension, min/max score, evidence status.	Returns items, total, filters and methodology version.
GET /api/v1/villages/{id}	Village profile and component evidence.	404 for unknown ID; includes missing indicators and sources.
GET /api/v1/map	GeoJSON features for visible villages.	Only valid geometries; properties include score/category/confidence.
GET /api/v1/rankings	Sorted priority list with factor explanations.	Deterministic tie-break on village ID.
POST /api/v1/scenarios	Calculate hypothetical score from user overrides.	Validate 0–100 overrides; return baseline and scenario side-by-side.
POST /api/v1/imports/csv	Validate/import data; admin-only if auth enabled.	Return accepted/rejected counts and row-level errors.
GET /api/v1/export.csv	Export current filtered dataset.	CSV columns include provenance, score version and coverage.
GET /api/v1/methodology	Weights, formulas, thresholds and limitations.	Versioned response; UI renders it.

6. Canonical API examples

```
GET /api/v1/villages/vil-001 { "id": "vil-001", "name": "Demo Village A", "location": {"lat": 17.2, "lon": 78.4}, "population": {"value": 4820, "year": 2021, "source": "demo"}, "score": {"overall": 51.4, "category": "vulnerable", "coverage_percent": 80, "config_version": "v1"}, "components": [ {"dimension": "digital", "score": 52, "evidence_type": "simulated", "source_date": "2026-10-01"}, {"dimension": "electricity", "score": 71, "evidence_type": "simulated", "source_date": "2026-10-01"} ], "gaps": [{"indicator": "broadband", "availability": 95, "reliability": 52, "gap_points": 43}], "data_notice": "DEMO DATA — NOT OFFICIAL MEASUREMENTS" }
```

The example above is illustrative schema only; do not reuse the example values as real-world measurements.

7. Scoring implementation detail

Represent each indicator with a normalization rule: direction (higher/lower better), lower bound, upper bound, target/threshold, unit, observation window, and missing-value policy. Normalization must return both score and status, e.g. valid, missing, stale, invalid, or out-of-range.

```
higher_is_better(x, min, max) = clamp(100 * (x - min) / (max - min), 0, 100) lower_is_better(x, min, max) = clamp(100 * (max - x) / (max - min), 0, 100) component_score = weighted_mean(valid_normalized_indicators) overall_score = weighted_mean(valid_component_scores, using configured dimension weights) coverage_percent = 100 * sum(weights of valid dimensions) / sum(all dimension weights)
```

These formulas are simple defaults, not universal truths. For indicators such as road passability or service uptime, use meaningful engineering/policy thresholds when available. Store raw values so the scoring rules can be recalculated without losing source data.

8. Gap analysis rules

• Availability and reliability must describe the same service, geographic unit and compatible period.

• Normalize both to the same scale before subtracting.

• gap_points = availability_score – reliability_score.

• Positive gap: availability is higher than observed reliability; negative gap: performance indicator is higher than availability proxy and requires review.

• If source dates differ beyond configured tolerance, display “date mismatch” and suppress the numeric gap or mark it low-confidence.

• If either side is missing, display “not enough evidence”; do not infer a gap.

9. Priority ranking design

Compute a transparent need score from normalized factors. Recommended initial factors: infrastructure deficit 35%, estimated population affected 25%, criticality 20%, lack of alternatives 10%, evidence confidence 10%. A product review should consider whether confidence should be an eligibility/verification gate instead of a positive priority factor; uncertain records must not be promoted merely because uncertainty is high. Show the individual factors and final weighted calculation.

10. Intervention simulation

• Input: selected village, baseline score version, chosen dimension(s), scenario score overrides, optional estimated cost and user-authored assumptions.

• Process: validate override values, recompute score and coverage, recalculate category and priority, compare baseline to scenario.

• Output: baseline score, scenario score, absolute change, category change, assumed cost, improvement per cost unit (only when cost is supplied), assumptions and warning that this is not a causal forecast.

• Isolation: scenario state is stored separately and cannot update source observations.

• Testing: no-op scenario returns same score; improving one dimension changes the score by the mathematically expected amount; invalid overrides return 422.

11. Frontend information architecture

Page	Key components	Primary interaction
Overview	Summary cards, distribution chart, high-priority list, data freshness notices.	Click KPI to apply a filter.
Map explorer	Map, legend, filters, searchable village table, selected-village panel.	Map/list selection synchronization.
Village detail	Score, dimension breakdown, gap cards, population/service access, source table, priority reasons.	Switch between overview and raw evidence.
Priority ranking	Sortable ranking table, factor breakdown, data-confidence indicator.	Sort/filter/export.
Scenario lab	Baseline vs scenario comparison, dimension controls, optional cost input, assumptions box.	Run scenario/reset to baseline.
Methodology & sources	Formula, weights, category definitions, data dictionary, source links, limitations.	Change config only in admin/dev mode.

12. Design system

• Use a neutral black/white/gray design if following the team's presentation preference; color may be used only for map categories if the team later approves it, and categories must also use text/icons so color is not the only signal.

• Use consistent spacing, typography hierarchy, tabular numerals for scores, clear empty states and visible focus.

• Never imitate official government branding or use government logos without permission.

• Charts need titles, units, legends and accessible text summaries.

• Use loading, error, empty and stale-data states on every data-driven page.

13. Data ingestion and validation pipeline

```
Upload → file/type/size validation → required column mapping → data type checks → coordinate and numeric range checks → duplicate handling → provenance required → preview accepted/rejected rows → explicit confirmation → persist observations → recalculate scores → record import audit event.
```

Reject or quarantine invalid rows rather than silently coercing them. Keep the original file hash/name and import timestamp where appropriate, but do not retain sensitive files unnecessarily.

14. Deployment and environment

Variable	Purpose	Example / handling
DATABASE_URL	Database connection.	Set in host secret store; never commit real value.
APP_ENV	local/staging/production.	Default local.
CORS_ORIGINS	Allowed frontend origins.	Exact deployed frontend URL.
DATA_MODE	demo or configured-source.	Demo mode clearly labels simulated metrics.
LOG_LEVEL	Server log verbosity.	INFO in deployment; avoid sensitive payloads.
OPTIONAL_AI_API_KEY	Only if a real AI explanation service is enabled.	Server-side only; deterministic explanation works without it.

15. Observability and failure handling

• GET /health checks API process and required dependencies.

• Log import IDs, counts and validation errors without storing sensitive report text in logs.

• Return consistent error objects: code, message, details, request_id.

• Frontend must handle API unavailable, empty dataset, partial coverage and map-library failure with a usable table view.

• Core scoring and simulation must not fail when an external geocoder or AI API is unavailable.

16. Architecture decisions

Decision	Rationale	Trade-off
Deterministic weighted scoring for MVP	Explainable and testable with limited data.	Not a validated predictive model; weights need calibration.
React + FastAPI	Fast to build, familiar tooling, clear separation.	Two deployables to configure.
GeoJSON/Leaflet first	Low setup overhead for small prototype geography.	Large-scale geometry needs simplification/tiling.
Postgres/PostGIS for growth	Spatial queries and structured history.	SQLite is faster to start locally.
Provenance-first observations	Supports trust, audit and future integrations.	More schema work at start, avoids future data confusion.

17. Source landscape and novelty boundary

Gram Manchitra already offers Gram Panchayat GIS, asset and institutional layers, proximity tools and Mission Antyodaya reports. Mission Antyodaya collects village/GP indicators across multiple development sectors and creates rankings/gap reports. BharatNet is a rural connectivity programme. TRAI MySpeed measures network quality such as speed, latency, packet loss and jitter. The World Bank Rural Access Index measures rural population access to all-season roads. These are complementary data/context sources and prior art, not systems to dismiss.

The defensible contribution for InfraPulse is the integrated product workflow: provenance-aware multi-sector observations; transparent service reliability scoring; explicit availability-performance gap; population and critical-service impact; uncertainty-aware priority ranking; and user-controlled scenario comparison. This is a proposed differentiation, not a claim of global uniqueness. Validate it through a feature-by-feature review before any formal novelty claim.

Gram Manchitra | Mission Antyodaya | BharatNet | TRAI MySpeed | World Bank RAI

18. Architecture completion checklist

• Domain logic has no dependency on HTTP request objects or UI code.

• All score and priority configuration is versioned.

• Every observation retains provenance and status.

• API schemas are documented and validated.

• Map has a table fallback and handles missing geometry.

• Scenario calculations are isolated from baseline data.

• Import and scoring logic have automated tests.

• Deployment uses environment variables and includes a health endpoint.